

1 L2–3 Supervised Learning

1.1 Classification and Loss

ERM: population risk $\theta^* = \arg \min_{\theta} \mathbb{E}_{x \sim p} \text{err}(f(x; \theta), y(x))$; empirical risk $\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_i \text{err}(f(x_i; \theta), y_i)$. **Binary logistic:** $p(y = 1|x) = \sigma(w^\top x + b)$,

$$\ell = -y \log p - (1 - y) \log(1 - p), \quad \frac{\partial \ell}{\partial z} = p - y.$$

Multi-class: logits $z \in \mathbb{R}^K$, $p_k = e^{z_k} / \sum_j e^{z_j}$,

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_j e^{z_j}, \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y].$$

One-hot CE: $-\sum_k y_k \log p_k$.

1.2 MLP and Backprop

Layer: $h^{(0)} = x$, $z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$, $h^{(\ell)} = \phi(z^{(\ell)})$. Batch code convention often uses $X \in \mathbb{R}^{B \times n}$, $Z = XW + b$, $W \in \mathbb{R}^{n \times m}$. **Backprop:** define $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$:

$$\partial_{W^{(\ell)}} L = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \partial_{b^{(\ell)}} L = \delta^{(\ell)},$$

$$\delta^{(\ell-1)} = ((W^{(\ell)})^\top \delta^{(\ell)}) \odot \phi'(z^{(\ell-1)}).$$

Activation: sigmoid/tanh saturate; ReLU $\max(0, z)$ keeps positive-side gradient; ELU/smooth variants improve negative side. **Subgradient:** ReLU at 0 uses any value in $[0, 1]$ by convention.

1.3 CNN

Motivation: locality + weight sharing. Convolution:

$$y_{i,j,\rho} = \sum_{u,v,c} K_{u,v,c,\rho} x_{i+u,j+v,c} + b_{\rho}.$$

For input $H \times W \times C_{\text{in}}$, kernel $K_h \times K_w$, padding P , stride S :

$$H' = \lfloor (H + 2P - K_h) / S \rfloor + 1, \quad W' = \lfloor (W + 2P - K_w) / S \rfloor + 1.$$

Params $K_h K_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$. **Scanning MLP:** same detector slides across space. **Pooling:** max/avg reduces spatial size and jitter. **Padding:** keeps boundary info; same padding for odd kernel uses $P = (K - 1) / 2$.

1.4 Optimization

GD/SGD: full-batch GD uses all data gradient; SGD uses one noisy sample; mini-batch uses B samples to trade variance vs. throughput. $x_{k+1} = x_k - \eta_k \nabla f(x_k)$. L -smooth means gradient Lipschitz:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \|\nabla^2 f(x)\|_{\text{op}} \leq L \text{ (C}^2\text{)}.$$

For convex f , $0 \leq \nabla^2 f(x) \leq LI$. Smooth upper bound:

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2,$$

Strong convexity: $f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|y - x\|^2$; with L -smooth and C^2 , $\mu I \preceq \nabla^2 f(x) \preceq LI$. GD has linear convergence $k = O(\kappa \log(1/\epsilon))$, $\kappa = L/\mu$. Thus $0 < \eta < 2/L$ decreases the smooth upper bound. **Newton:** from quadratic Taylor,

$$x_{k+1} = x_k - \eta [\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

g_k = gradient on x_k . **Momentum:** $\Delta x^k = \beta \Delta x^{k-1} - \eta \nabla f(x^k)$, $x^{k+1} = x^k + \Delta x^k$. **Nesterov:** compute gradient at lookahead $x_k + \beta(x_k - x_{k-1})$. **Adaptive LR:** rescale each coordinate, helping ill-conditioned optimization. **AdaGrad:** $s_k = s_{k-1} + g_k^2$, $x_{k+1} = x_k - \eta g_k / (\sqrt{s_k} + \epsilon)$; rarely updated dims have smaller s_{k_j} and larger effective LR; may decay too fast. **RMSProp:** $s_k = \rho s_{k-1} + (1 - \rho) g_k^2$. **Adam:**

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2,$$

$$x_{k+1} = x_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon), \quad \hat{m}_k = m_k / (1 - \beta_1^k).$$

1.5 Regularization and Architectures

Dropout: randomly zero units, prevents reliance on particular neurons/weights, approximates ensemble and implicit regularization (often L2-like). Scaling: inverted dropout divides by keep prob α at train and leaves inference unchanged; standard dropout leaves train unchanged and multiplies activations by α at inference. **Early stopping:** stop when held-out validation worsens while training keeps improving. **Weight decay:** L2 regularization, $L_{\text{reg}}(w) = L_{\text{loss}}(w) + \alpha \|w\|_2^2$. **PCA color jitter:** add $P(\alpha \times \lambda)$ to RGB channels, $\alpha_i \sim \mathcal{N}(0, 0.1)$, simulating lighting/color changes. **Gradient clipping:** $g \leftarrow g \min(1, \tau / \|g\|)$.

BatchNorm: $\mu_B = \frac{1}{B} \sum_i z_i$, $\sigma_B^2 = \frac{1}{B} \sum_i (z_i - \mu_B)^2$, $z_i = \gamma \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$.

BN backward: if $u_i = (z_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then $\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial u_i} \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} +$

$$\frac{\partial L}{\partial \sigma_B^2} \frac{2(z_i - \mu_B)}{B} + \frac{\partial L}{\partial \mu_B} \frac{1}{B}.$$
 BN inference: use running stats $\mu_{\text{run}} \approx$

$$\frac{1}{N_{\text{batch}}} \sum_b \mu_B^{(b)}, \sigma_{\text{run}}^2 \approx \frac{1}{N_{\text{batch}}} \sum_b \frac{B}{B-1} \sigma_B^{(b)}.$$

LayerNorm: normalize hidden dimension inside one sample; better for sequence models. **ResNet:** $h_{\ell+1} = h_{\ell} + F(h_{\ell})$, residual path eases optimization. **DenseNet:** concatenate previous features. **FCN:** replace fully connected layer by convolution for dense prediction.

2 L4 Energy-Based Models

2.1 Energy View

Generative modeling: learn $P(X)$ or $P(X, y) = P(y)P(X|y)$ instead of $P(y|X)$. **EBM:** unnormalized score via energy

$$p_{\theta}(x) = \frac{e^{-E_{\theta}(x)}}{Z_{\theta}}, \quad Z_{\theta} = \int e^{-E_{\theta}(x)} dx.$$

Low energy means high probability; training and inference both need sampling or partition-function gradients. $\log p_{\theta}(x) = -E_{\theta}(x) - \log Z_{\theta}$, $\nabla_{\theta} \log p_{\theta}(x) = -\nabla_{\theta} E_{\theta}(x) - \frac{1}{Z_{\theta}} \nabla_{\theta} \left(\sum_{x'} e^{-E_{\theta}(x')} \right) = -\nabla_{\theta} E_{\theta}(x) + \mathbb{E}_{x' \sim p_{\theta}} [E_{\theta}(x')]$.

2.2 Hopfield Network

State $y_i \in \{-1, 1\}$, local field $h_i = \sum_{j \neq i} w_{ij} y_j + b_i$. Update flips if $y_i h_i < 0$.

$$E(y) = -\sum_{i < j} w_{ij} y_i y_j - \sum_i b_i y_i = -\frac{1}{2} y^\top W y - b^\top y.$$

Asynchronous flip changes energy by $\Delta E = 2y_i h_i$; if $y_i h_i < 0$, energy decreases. Finite states imply convergence to a local minimum. **Hebbian storage:** for K patterns, $w_{ij} = \frac{1}{K} \sum_{p=1}^K y_i^p y_j^p$, $w_{ii} = 0$; stability sufficient condition $K \leq N / (4 \log N)$ for random patterns. Multiple patterns cause **spurious minima**; capacity is limited, roughly linear in neuron count for stable random memories.

Objective for W : make desired patterns valleys and undesired states high energy. For $E(y) = -\frac{1}{2} y^\top W y$,

$$W = \arg \min_W \left(\sum_{y \in P} E(y) - \sum_{y' \notin P} E(y') \right),$$

$$W \leftarrow W + \eta \left(\mathbb{E}_{y \in P} [y y^\top] - \mathbb{E}_{y' \notin P} [y' y'^\top] \right).$$

Cannot enumerate exponential state space, so sample $y \in P$, initialize y' from a random desired pattern, run 2–4 dynamics steps, update with $y y^\top - y' y'^\top$.

2.3 Boltzmann Machine

Stochastic Hopfield: energy $E(y)$ as Hopfield, $p(y) = \frac{1}{Z} e^{-E(y)} \propto e^{-E(y)}$. Conditional Gibbs update for $\{-1, 1\}$ units:

$$h_i = \sum_j w_{ij} y_j + b_i, \quad \log \frac{P(y_i = 1 | y_{-i})}{P(y_i = -1 | y_{-i})} = 2h_i, \quad P(y_i = 1 | y_{-i}) = \sigma(2h_i).$$

Stochastic evolution: initialize $y^{(0)}$, Gibbs-sample coordinates $y_i(t+1) \sim \text{Bernoulli}(\sigma(z_i(t)))$ for $P(y_i = 1)$; recover stored pattern from tail samples by $\hat{y}_i = \frac{1}{N} \sum_{t=L-M+1}^L y_i(t) > 0$ (majority/sign). **Annealing:** $p_T(y) \propto e^{-E(y)/T}$, sample $y_i(T) \sim \text{Bernoulli}(\sigma(z_i(T)/T))$, then $T \leftarrow \alpha T$; lower T shifts exploration to exploitation.

BM Training: for $E(y) = -\frac{1}{2} y^\top W y$,

$$P(y) = \frac{e^{\frac{1}{2} y^\top W y}}{\sum_{y'} e^{\frac{1}{2} y'^\top W y'}}, \quad L(W) = \frac{1}{|P|} \sum_{y \in P} \frac{1}{2} y^\top W y - \log \sum_{y'} e^{\frac{1}{2} y'^\top W y'}.$$

$$\nabla w_{ij} L = \frac{1}{|P|} \sum_{y \in P} y_i y_j - \mathbb{E}_{p_{\theta}} [y_i y_j] \approx \frac{1}{|P|} \sum_{y \in P} y_i y_j - \frac{1}{M} \sum_{y' \in S} y'_i y'_j.$$

First term: compute average over P . Second term: sample from p_{θ} , compute average. **Sample method:** random initialize $y(0)$, cycle over i (coordinates) and sample from $P(y_i(t) | y_{-i}(t))$, after convergence select the final M states as samples. **Updating:** $w_{ij} \leftarrow w_{ij} + \eta \nabla w_{ij} L(W)$. Learning = **positive phase** data correlation minus **negative phase** model correlation.

With hidden neurons: $y = (v, h)$

$$\nabla w_{ij} L(W) = \frac{1}{|P|} \sum_{v \in P} \mathbb{E}_{h|v} [y_i y_j] - \mathbb{E}_{y' \sim P_W} [y'_i y'_j].$$

Boltzmann for classification: $y = (v, h, c)$, c : visible, one-hot vector

2.4 RBM and Contrastive Divergence

RBm: visible v and hidden h form a bipartite graph; no visible-visible or hidden-hidden edges:

$$E(v, h) = -v^\top W h - b^\top v - c^\top h.$$

Layer-wise Gibbs updates factorize, with $\sigma(z) = 1 / (1 + e^{-z})$:

$$z_j^h = c_j + \sum_i W_{ij} v_i, \quad P(h_j = 1 | v) = \sigma(z_j^h),$$

$$z_i^v = b_i + \sum_j W_{ij} h_j, \quad P(v_i = 1 | h) = \sigma(z_i^v).$$

Iterative sampling: $v^{(0)} \rightarrow h^{(0)} \rightarrow v^{(1)} \rightarrow h^{(1)} \rightarrow \dots$; after mixing, $(v^{(\infty)}, h^{(\infty)}) \sim P_W$. With K hidden samples per data and M free samples:

$$\hat{\nabla} W_{ij} L(W) = \frac{1}{N_P K} \sum_{v \in P} \sum_{k=1}^K v_i^{(0)} h_{jk}^{(0)} - \frac{1}{M} \sum_{m=1}^M v_{im}^{(\infty)} h_{jm}^{(\infty)}.$$

CD-k: start at data $v^{(0)}$, run k Gibbs steps, update $W \propto v^{(0)} h^{(0)} - v^{(k)} h^{(k)}$. **CD-1/short CD** replaces the long negative chain by one/few reconstructions:

$$\nabla W_{ij} L \approx \frac{1}{N_P} \sum_{v \in P} \left(v_i^{(0)} h_j^{(0)} - v_i^{(1)} h_j^{(1)} \right).$$

2.5 Sampling

Inverse CDF: sample $u \sim U[0, 1]$, choose category by cumulative mass. **Box-Muller:** Gaussian from uniform:

$$z_1 = \sqrt{-2 \log u_1} \cos(2\pi u_2), \quad z_2 = \sqrt{-2 \log u_1} \sin(2\pi u_2).$$

Importance sampling: $\mathbb{E}_P[f(x)] = \mathbb{E}_q[f(x)p(x)/q(x)]$. Good q must cover p 's high-mass region. **MCMC: Detailed balance** $\pi(x)T(x \rightarrow x') = \pi(x')T(x' \rightarrow x)$ implies stationarity of π . **Ergodic:** intuitively, any desired state can be visited with positive probability, giving a unique stationary distribution. Uniform minorization

$$\min_{z, z'}: \pi(z') > 0 \quad \frac{T(z \rightarrow z')}{\pi(z')} = \delta > 0$$

is a strong sufficient condition for uniform ergodicity. **Metropolis Hastings Algo.:** construct Markov chain with stationary π . Current x , draw proposal $x' \sim q(x'|x)$, transition to x' with prob. $\alpha = \min \left(1, \frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)} \right)$, otherwise stays at x . Choice of q : random proposal $q(x'|x)$ = noise (e.g. Gaussian, then it relies on $|x - x'|$), then q terms in α vanishes.

For EBM, partition Z cancels.

Gibbs: sample a single coordinate per step. As MH, use $q(s_i \rightarrow s'_i) = p(s'_i | s_{j \neq i})$, so acceptance ratio is always 1.

SGLD: $x_{t+1} = x_t - \frac{\eta}{2} \nabla_x E_{\theta}(x_t) + \sqrt{\eta} \xi_t$.

3 L5 Generative Adversarial Networks

3.1 Implicit Generative Model

Instead of density $p_{\theta}(x)$, learn sampler $x = G_{\theta}(z)$, $z \sim p_z$. Likelihood-free learning. Need differentiable distance between generated and real distributions. GAN learns this distance with a discriminator.

GAN: generator $G_{\theta}(z)$, $z \sim p(z) = N(0, I)$; discriminator $D_{\phi}(x)$, classify if the data is from p_{data} or G_{θ} . Objective: $L = \min_{\theta} \max_{\phi} V(D_{\phi}, G_{\theta})$,

$$V(D_{\phi}, G_{\theta}) = \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\phi}(x) + \mathbb{E}_{z \sim p_z} \log(1 - D_{\phi}(G_{\theta}(z))).$$

Dataset: $\{(x, 1) : x \sim p_{\text{data}}\} \cup \{(\hat{x}, 0) : \hat{x} \sim G_{\theta}(z)\}$.

Train D : $L(\phi) = \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\phi}(x) + \mathbb{E}_{z \sim p_z} \log(1 - D_{\phi}(G(z)))$.

Train G : $L(\theta) = \mathbb{E}_{z \sim p(z)} [\log D_{\phi}(G_{\theta}(z))]$.

For fixed G , the optimal D is $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$. Substitute:

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G), \quad \text{JSD: } \text{JSD}(p \| q) = \text{KL}(p \| \frac{p+q}{2}) + \text{KL}(q \| \frac{p+q}{2}), \geq 0, \sqrt{\text{JSD}} \text{ satisfies triangular inequality}$$

Non-saturating generator loss $-\mathbb{E}_z \log D(G(z))$ gives stronger gradients.

3.2 Evaluation and Failure Modes

Mode collapse: generator covers few modes (converges to a few samples). Intrinsic issue of JSD. **Sample diversity:** for multimodal p , forward KL is mode-covering, while reverse KL and JSD can be mode-seeking/collapse:

$$\text{KL}(p \| q) = \mathbb{E}_p \log \frac{p}{q}, \quad \text{KL}(q \| p) = \mathbb{E}_q \log \frac{q}{p},$$

$$\text{JSD}(p \| q) = \frac{1}{2} \text{KL}(p \| m) + \frac{1}{2} \text{KL}(q \| m), \quad m = \frac{p+q}{2}.$$

Training instability: moving target game can oscillate; if D 's task is too simple/strong, G gets vanishing or weak gradients. **Inception Score:**

$$\text{IS} = \exp \left(\mathbb{E}_{x \sim G} \text{KL}(f(y|x) \| p_f(y)) \right), \quad p_f(y) = \mathbb{E}_{x \sim G} [f(y|x)].$$

Since $\mathbb{E}_x \text{KL}(f(\cdot|x) \| p_f) = -\mathbb{E}_x H(f(\cdot|x)) + H(p_f)$, high IS needs confident images and diverse classes. IS never uses p_{data} , so it cannot detect memorization and is blind to intra-class artifacts. **FID:** use Inception-v3 pool3 2048-d features; estimate (μ_P, Σ_P) for p_{data} and (μ_G, Σ_G) for p_G :

$$\text{FID} = \|\mu_P - \mu_G\|_2^2 + \text{tr} \left(\Sigma_P + \Sigma_G - 2(\Sigma_P \Sigma_G)^{1/2} \right).$$

Lower FID is better.

3.3 GAN Training Techniques

Feature Matching: train G to match statistics of discriminator features instead of only fooling D :

$$L_G(\theta) = \left\| \mathbb{E}_{\hat{x} \sim G_{\theta}} [f(x; \phi)] - \mathbb{E}_{x \sim p_{\text{data}}} [f(x; \phi)] \right\|_2^2.$$

Here $f(x; \phi)$ is an internal feature of D , often the final activation layer before classification. **Other tricks:** One-Sided Label Smoothing; Historical Averaging; Minibatch Discrimination; Virtual Batch Normalization.

3.4 WGAN Family

JSD can be flat when supports do not overlap. Wasserstein-1: $W(p, q) = \sup \|f\|_{L \leq 1} \mathbb{E}_{x \sim p} f(x) - \mathbb{E}_{x \sim q} f(x)$.

WGAN: critic f_{ϕ} , no D . WGAN objective:

$$\min_G \max_{f_{\phi}: \|f_{\phi}\|_{L \leq 1}} \mathbb{E}_{x \sim p_{\text{data}}} [f_{\phi}(x)] - \mathbb{E}_{\hat{x} \sim p_G} [f_{\phi}(\hat{x})].$$

Equivalently, the critic maximizes $L(\phi) = \mathbb{E}_{x \sim p_{\text{data}}} [f_{\phi}(x)] - \mathbb{E}_{\hat{x} \sim p_G} [f_{\phi}(\hat{x})]$, while G_{θ} minimizes $L(\theta) = \mathbb{E}_{x \sim G_{\theta}(z)} [f_{\phi}(x)]$.

Lipschitz constraint: weight clipping, spectral norm, or **WGAN-GP:** sample $\tilde{x} \leftarrow (1 - \epsilon)x + \epsilon \tilde{x}$, $\epsilon \sim \text{Unif}(0, 1)$, between real x and fake $\tilde{x}$:

$$\max_{\phi} \mathbb{E}_{x \sim p_{\text{data}}} f_{\phi}(x) - \mathbb{E}_{\hat{x} \sim p_G} f_{\phi}(\hat{x}) - \lambda \mathbb{E}_{\tilde{x}} \left(\|\nabla_{\tilde{x}} f_{\phi}(\tilde{x})\|_2 - 1 \right)^2.$$

If written as a critic minimization loss, the gradient penalty has positive sign. **DCGAN:** conv/deconv, batchnorm, stable architecture. **BigGAN:** scale + class conditioning + truncation trick. **GigaGAN:** text-to-image GAN scaling. **R3GAN:** regularized robust objective.

3.5 Variants and Applications

Conditional GAN: $G(z,c), D(x,c)$. **Semi-supervised GAN:** discriminator has $K+1$ classes, K real labels plus fake. **BiGAN/ALI:** learn encoder $E(x)$ with discriminator over (x,z) pairs. Idea: $(z,G(z))$ and $(E(x),x)$ from same distribution. gives representation learning. **Pix2Pix:** paired image-to-image translation with conditional GAN + reconstruction loss. **CycleGAN:** unpaired translation using cycle consistency $G: X \rightarrow Y, F: Y \rightarrow X, \|F(G(x)) - x\| + \|G(F(y)) - y\|$. **DragGAN:** interactive image manipulation by optimizing latent/code with handle-point constraints.

4 L6 Flow and VAE

4.1 Normalizing Flow

Invertible differentiable generator $x = G_\theta(z)$, transform a simple distribution to the data distribution. $\log p_X(x) = \log p_Z(G^{-1}(x)) + \log \left| \det \frac{\partial G^{-1}}{\partial x} \right|$. Composition: $z_K = G(z_0) = f_K \circ \dots \circ f_1(z_0)$, $\log p_X(z_K) = \log p_Z(z_0) - \sum_k \log \left| \det \frac{\partial f_k}{\partial z_k} \right|$. Flow design needs invertible layer, efficient inverse, efficient compute determinant of Jacobi. **Coupling:** split $x = (x_a, x_b)$, to (y_a, y_b) , $y_a = x_a, y_b = x_b \odot e^{s(x_a) + t(x_a)}$, $\log |\det J| = \sum s(x_a)$. s, t : learnable. sometimes needs premutaion among coordinates or linear transformation. **Glow:** invertible 1×1 convolution mixes channels. Since the same channel-mixing matrix W is applied at each spatial location,

$$\log \left| \det \frac{d \text{ conv2D}(h; W)}{dh} \right| = H_{\text{sp}} W_{\text{sp}} \log |\det W|.$$

LU parameterization $W = PL(U + \text{diag}(s))$ makes $\log |\det W| = \sum_i \log |s_i|$ cheap. **Dequantization:** add uniform noise to discrete pixels so continuous density is well-defined. **Continuous NF / Neural ODE:** $\frac{dz(t)}{dt} = f_\theta(z(t), t)$, $\log p(z(t_1)) = \log p(z(t_0)) - \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial z(t)} \right) dt$.

4.2 GMM and EM

Latent-variable likelihood $\log p_\theta(x) = \log \sum_z p_\theta(x, z)$. **Generic EM:** E-step estimates/posteriorizes latent variables under old parameters; M-step maximizes expected complete-data log-likelihood. **GMM responsibility:** for K mixture components,

$$\gamma_{ij} := p(z_i = j | x_i) = \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}.$$

GMM EM lower bound:

$$\sum_i \log \sum_j \gamma_{ij} \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\gamma_{ij}} \geq \sum_i \sum_j \gamma_{ij} \log \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\gamma_{ij}}.$$

E-step fixes parameters and optimizes γ :

$$\gamma^{(t+1)} = \arg \max_{\gamma_{ij} \geq 0, \sum_j \gamma_{ij} = 1} \sum_{i,j} \gamma_{ij} \log \pi_j^{(t)} \mathcal{N}(x_i | \mu_j^{(t)}, \Sigma_j^{(t)}) - \sum_{i,j} \gamma_{ij} \log \gamma_{ij}.$$

M-step fixes γ and optimizes parameters:

$$(\pi, \mu, \Sigma)^{(t+1)} = \arg \max_{\pi, \mu, \Sigma} \sum_{i,j} \gamma_{ij}^{(t+1)} \log \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j).$$

4.3 Evidence Lower Bound (ELBO)

For latent-variable models, insert any inference distribution $q_\phi(z|x)$ and apply Jensen:

$$\begin{aligned} \log p_\theta(x) &= \log \int p(z) p_\theta(x|z) dz \\ &= \log \int q_\phi(z|x) \frac{p(z) p_\theta(x|z)}{q_\phi(z|x)} dz \\ &\geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)). \end{aligned}$$

Gap: $\log p_\theta(x) - \text{ELBO} = \text{KL}(q_\phi(z|x) \| p_\theta(z|x))$.

4.4 VAE

Parametrize encoder $q_x(z) = \mathcal{N}(\mu_\theta(x), \sigma_\theta(x))$, decoder $p_\psi(x|z) = \mathcal{N}(\psi(z), \beta I)$, prior $p(z)$.

$$\begin{aligned} \text{ELBO} &= \int q_x(z) \log p_\psi(x|z) dz - \text{KL}(q_x(z) \| p(z)) \\ &= -\frac{1}{2\beta} \mathbb{E}_{q_x(z)} \|\psi(z) - x\|_2^2 - \text{KL}(q_x(z) \| p(z)) \\ &= -\frac{1}{2\beta} \mathbb{E}_{x \sim \mathcal{N}(0,I)} \|\psi(\mu_\theta(x) + \sigma_\theta(x)\epsilon) - x\|_2^2 - \text{KL}(q_x(z) \| p(z)). \end{aligned}$$

Reparameterization: $z = \mu_\theta(x) + \sigma_\theta(x)\epsilon, \epsilon \sim \mathcal{N}(0,I)$. If $p(z) = \mathcal{N}(0,I)$, Gaussian KL:

$$\text{KL}(q_x(z) \| p(z)) = \frac{1}{2} \|\mu_\theta(x)\|_2^2 + \frac{1}{2} \sigma_\theta^2(x) - \log \sigma_\theta(x) + c.$$

Gaussian KL: for $p = \mathcal{N}(\mu_1, \sigma_1^2)$, $q = \mathcal{N}(\mu_2, \sigma_2^2)$,

$$\text{KL}(p \| q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_2^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}.$$

For d -dimensional Gaussians:

$$\begin{aligned} \text{KL}(\mathcal{N}_1 \| \mathcal{N}_2) &= \frac{1}{2} [\text{tr}(\Sigma_2^{-1} \Sigma_1) + (\mu_2 - \mu_1)^\top \Sigma_2^{-1} (\mu_2 - \mu_1) \\ &\quad - d + \log \frac{|\Sigma_2|}{|\Sigma_1|}]. \end{aligned}$$

Terms: reconstruction = data fidelity; KL = latent regularization. **β -VAE:** increase KL pressure to encourage disentangled/less-correlated latents,

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \beta \text{KL}(q_\phi(z|x) \| p(z)).$$

PixelVAE/NVAE: hierarchical latent variables improve global coherence and likelihood.

5 L7 Diffusion and Flow Matching

5.1 DDPM

Forward: add Gaussian noise with $\alpha_t = 1 - \beta_t$, $\tilde{\alpha}_t = \prod_{s=1}^t \alpha_s$:

$$\begin{aligned} q(x_t | x_{t-1}) &:= \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \\ q(x_t | x_0) &= \mathcal{N}(x_t; \sqrt{\tilde{\alpha}_t} x_0, (1 - \tilde{\alpha}_t) I). \end{aligned}$$

Reverse:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t), \quad p(x_T) = \mathcal{N}(0, I),$$

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I).$$

Loss / ELBO identity:

$$-\log p(x) = -\int q_\phi(z) \log p(x|z) dz + \text{KL}(q_\phi(z) \| p(z)) - \text{KL}(q_\phi(z) \| p(z|x)).$$

True posterior and Gaussian KL:

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I), \quad \tilde{\beta}_t = \frac{1 - \tilde{\alpha}_{t-1}}{1 - \tilde{\alpha}_t} \beta_t,$$

$$\tilde{\mu}_t = \frac{\sqrt{\tilde{\alpha}_{t-1}} \beta_t}{1 - \tilde{\alpha}_t} x_0 + \frac{\sqrt{\tilde{\alpha}_t} (1 - \tilde{\alpha}_{t-1})}{1 - \tilde{\alpha}_t} x_t.$$

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|_2^2 \right] + C.$$

Mean/noise prediction: $\mu_\theta(x_t, t) \approx \tilde{\mu}_t(x_t, x_0)$. With $x_t = \sqrt{\tilde{\alpha}_t} x_0 + \sqrt{1 - \tilde{\alpha}_t} \epsilon$,

$$\tilde{\mu}_t = \frac{1}{\sqrt{\tilde{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \epsilon \right), \quad \mu_\theta(x_t, t) = \frac{1}{\sqrt{\tilde{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

$$L_{t-1} = \mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \tilde{\alpha}_t)} \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2 \right] + C.$$

Training: input (x_t, t) , predict noise ϵ ; $\epsilon_\theta(x_t, t)$ is commonly a U-Net with timestep encoding. **Sampling:**

$$x_{t-1} = \frac{1}{\sqrt{\tilde{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

Start from $x_T \sim \mathcal{N}(0, I)$ and iterate to x_0 .

5.2 Score-Based Models

Score: learn score $\nabla_x \log p(x)$. **Langevin dynamics:** use only score. $x_{t+1} = x_t + \eta \nabla_x \log p(x_t) + \sqrt{2\eta} \epsilon_t, \epsilon_t \sim \mathcal{N}(0, I)$. continuous form: $dx_t = \nabla_x \log p(x_t) dt + \sqrt{2} dw_t, dw \sim \mathcal{N}(0, dt)$. **NCSN:** noise conditional score network. $x_t = x_0 + \sigma_t \epsilon$, learn $s_\theta(x_t, \sigma_t)$ to estimate $\nabla_x \log p(x_t)$. $x_t = \alpha_t x_0 + \sigma_t \epsilon$, score $s_\theta(x_t, t)$? MSE loss $L = \mathbb{E}_{x_t} \|\bar{s}_\theta(x_t, t) - \nabla_{x_t} \log p(x_t)\|_2^2$. $\mathbb{E}_{x_t} [\bar{s}_\theta(x_t, t)^\top \nabla_{x_t} \log p(x_t)] = \mathbb{E}_{x_t} [\nabla \cdot s_\theta(x_t, t)]$, using $\nabla \cdot (s p) = (\nabla \cdot s) p + s^\top \cdot \nabla p$

Denoising score matching: $\nabla_{x_t} \log p(x_t) = \mathbb{E}_{x_0 \sim p(\cdot|x_t)} [\nabla_{x_t} \log p(x_t|x_0)]$ $\mathbb{E}_{x_t} \|\bar{s}_\theta(x_t, t) - \nabla_{x_t} \log p(x_t|x_0)\|_2^2$, $L_{DSM} = \mathbb{E}_{x_0 \sim p_{data}, t \sim [0,1], x_t \sim q(x_t|x_0)} \|\bar{s}_\theta(x_t, t) - \nabla_{x_t} \log q(x_t|x_0)\|_2^2$, $q(x_t|x_0)$ is tractable. If $x_t = \alpha_t x_0 + \sigma_t \epsilon$, $\nabla_{x_t} \log q(x_t|x_0) = \nabla_{x_t} (-\frac{(x_t - \alpha_t x_0)^2}{2\sigma_t^2}) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$. Let $s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sigma_t}$, loss same as DDPM.

SDE: $dx = f(x, t) dt + g(t) dW_t$, reverse: $dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\tilde{W}_t$. A equivalent ODE (same distribution when same t): Probability-flow ODE: $dx = [f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x)] dt$. Likelihood along ODE: $\log p_0(x_0) = \log p_T(x_T) + \int_0^T \text{tr} \left(\frac{\partial f_\theta}{\partial x_t} \right) dt$. Inference: use learned score to solve SDE/ODE using a solver. Hence #sample-steps in training and inference can be different. DDIM: sample according to schedule

5.3 DPM-Solver and Applications

DPM solver 1 order DPM = DDIM.

DPM-Solver integrates diffusion ODE with high-order exponential integrators, reducing sampling steps. **Latent diffusion:** diffuse in autoencoder latent space, cheaper than pixel space. **DiT/SORA:** transformer over spacetime latent patches. **DALL-E 2:** CLIP prior + diffusion decoder. **DALL-E 3:** synthetic captions improve prompt following. **SDEdit:** add noise to input image then denoise under prompt. **ControlNet:** extra condition branch injects edges/depth/pose. **Logit-normal sampling:** biases timestep sampling toward informative middle region.

5.4 Flow Matching

Continuous flow $dx_t/dt = v_\theta(x_t, t)$. Objective: $\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \|v_\theta(x_t, t) - u_t(x_t)\|_2^2$. Conditional path from noise x_0 to data x_1 : $x_t = (1-t)x_0 + tx_1$, target $u_t = x_1 - x_0$. Conditional Flow Matching (CFM) samples pairs and regresses conditional velocity; marginal vector field is recovered in expectation. **Non-uniqueness:** many vector fields induce same marginals; choice affects training variance and sampling trajectory. **SD3:** flow target often parameterizes velocity between noise/data under continuous timestep.

6 L8–9 Sequence Modeling

6.1 Sequence Data and Tokenization

Sequence data includes language, audio, action; images/DNA/protein can be serialized. Autoregressive factorization: $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$. **Tokenization:** text $\rightarrow$ basic units $\rightarrow$ token IDs. Byte-Pair Encoding (BPE) repeatedly merges frequent adjacent symbols; balances vocabulary size and sequence length. **Embedding:** token ID $i \rightarrow e_i = W_E[i]$; positional info is needed because token embeddings alone are order-free.

6.2 Autoregressive CNN and Pixel Models

WaveNet: causal/dilated temporal convolution expands receptive field without recurrence. **PixelCNN:** image likelihood $p(x) = \prod_i p(x_i | x_{<i})$. under raster order; masked convolution prevents access to future pixels. Gated PixelCNN uses gates to improve expressiveness.

6.3 RNN and LSTM

RNN: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$, $y_t = W_{hy}h_t + b_y$. BPTT multiplies many Jacobians; singular values < 1 vanish, > 1 explode. **LSTM:** $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$, $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$, $\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$, $h_t = o_t \odot \tanh(c_t)$. Additive cell path gives controllable memory; forget gate decides retention.

6.4 Attention and Transformer

For token t : $q_t = W_Q x_t$, $k_j = W_K x_j$, $v_j = W_V x_j$, $a_{tj} = \frac{\exp(q_t^\top k_j / \sqrt{d_k})}{\sum_j \exp(q_t^\top k_j / \sqrt{d_k})}$, $y_t = \sum_j a_{tj} v_j$. Matrix form: $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$. **Multi-head:** parallel attention subspaces, concatenate and project. **Causal mask:** block future tokens. **Transformer block:** attention + MLP + residual + normalization. Complexity $O(T^2 d)$; FlashAttention reduces memory IO, Mamba/SSM explores linear-time sequence mixing.

6.5 Objectives and Case Studies

Objectives: autoregressive LM, masked/denoising, contrastive predictive. **Recipe:** pretraining, fine-tuning, low-rank tuning. **GPT:** decoder-only causal Transformer; next-token pretraining gives in-context learning with scale. **ViT:** image patches as tokens. **Swin:** shifted local windows for hierarchical vision. **CLIP:** contrastive image-text alignment,

$$L = -\frac{1}{2} \left[\frac{1}{N} \sum_i \log \frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}} + \frac{1}{N} \sum_i \log \frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ji}/\tau}} \right].$$

SigLIP: pairwise sigmoid loss avoids global softmax. **Whisper:** audio Transformer trained on large-scale speech. **MAR:** masked autoregressive modeling for images.

7 L10 Modern Large Models

7.1 Pretraining, Scaling, Prompting

Pipeline: Pretraining $\rightarrow$ Base GPT $\rightarrow$ SFT $\rightarrow$ RLHF $\rightarrow$ ChatGPT-like. Pre-training builds capability by next-token prediction; post-training reshapes behavior into instruction-following. Base GPT tends to continue text; chat model tries to answer the user. **Scaling laws:** held-out loss decreases smoothly as model size N , data D , and compute C grow, approximately by power law. The important lesson is not the exact constants but bottleneck balance: scale parameters, tokens, and compute together. Useful heuristics: data should grow with model size ($D \propto N^{0.74}$); compute-optimal training spends extra compute mostly on larger N , somewhat on batch B , and barely on serial steps S . **Prompting:** zero-shot/few-shot puts task description/examples in context without parameter updates. GPT-3 made in-context learning practical; instruction tuning makes this behavior more reliable by training on natural-language tasks.

7.2 SFT and RLHF

SFT: learn from human demonstrations (x, y) by next-token imitation:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{\mathcal{D}_{\text{SFT}}} \sum_t \log \pi_\theta(y_t | x, y_{<t}).$$

It teaches desired response style but only uses positive examples, so it cannot directly say why one acceptable answer is better than another. **Reward model:** pairwise preference $y_w \succ y_l$ teaches a scalar reward $r_\phi(x, y)$:

$$P_\phi(y_w \succ y_l | x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

$$\mathcal{L}_{\text{RM}} = -\mathbb{E} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

RLHF: optimize the policy for reward while staying near a reference/SFT model:

$$\max_\theta \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[r_\phi(x, y) - \beta \text{KL}(\pi_\theta(\cdot|x) \| \pi_{\text{ref}}(\cdot|x)) \right].$$

Why KL: prevents reward hacking and language-quality drift. **Pretrain vs post-train:** pretraining compresses web text distribution; post-training optimizes human utility/preference over responses.

7.3 Hallucination, RAG, Reasoning

Hallucination: SFT can teach a confident answer even when the base model lacks knowledge. Preference/RLHF can reward uncertainty awareness, e.g. prefer "I don't know" over wrong entities. Negative samples (x, y^+, y^-) are valuable because they mark bad-but-plausible behavior. **RAG:** combine parametric memory in θ with retrieved non-parametric memory z :

p(y | x) ≈ ∑_{z ∈ TopK(x)} p_η(z | x) p_θ(y | x, z).

RAG helps latest knowledge without retraining, but retrieval is not the same as reasoning. **LRM:** reasoning model spends test-time compute on internal thinking before final answer:

x → z_{1:m} → y.

CoT prompting elicits intermediate reasoning through prompt examples; reasoning RL trains the behavior with verifier reward. Thinking labels are not unique, so RL can reward final correctness without prescribing one chain. Trajectory:

τ = (z_1, ..., z_m, y_1, ..., y_T), R(x, τ) = { +1, V(x, y) = correct, -1, V(x, y) = incorrect,

Agent loop: reasoning can extend to environment interaction with actions:

s_t = (x, memory, observations_{<t}), a_t ∈ {search, browse, code, write, ...}.

7.4 Multimodal Models

Grounding: language-only models condition on text; multimodal models condition on text plus image/video/audio/interface inputs:

p_θ(y | x_text, x_image, x_video, ...).

Flamingo: extends GPT-style few-shot prompting to interleaved text-image/video input:

p_θ(y | t_1, v_1, t_2, v_2, ..., t_k, v_k, t_{k+1}).

Architecture: frozen vision encoder + frozen LM; Perceiver Resampler compresses many visual features into few visual tokens; gated cross-attention injects visual information while protecting frozen LM behavior. **LLaVA:** visual instruction tuning samples $(I, x_{\text{instruction}}, y_{\text{assistant}})$; CLIP features are projected into LM embedding space:

F_I = f_{\text{CLIP}}(I), Z_I = F_I W, [Z_I; E(x_1); ...; E(x_T)].

Two-stage training: first align visual features by training projection W ; then freeze vision encoder and tune projection + LM for assistant behavior. **Unified multimodal/BAGEL:** one decoder for understanding/generation/editing/reasoning on interleaved data $t_1, I_1, t_2, I_2, \dots, V_1, t_k$; ViT pathway supports understanding, VAE pathway supports generation/reconstruction. Emergence order: understanding → generation → editing → reasoning.